به نام خدا



دانشکده مهندسی کامپیوتر

درس سیستم عامل
نیم‌سال اول ۰۵-۰۶
استاد: دکتر اسدی

تمرین سری سوم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. با توجه به کتابخانه‌ی pthreads به سوالات زیر پاسخ دهید:

آ– قطعه کد زیر را در نظر بگیرید:

```c
#include <stdio.h>
#include <pthread.h>

int value = 10;

void* worker(void* arg) {
    int step = *(int*)arg;
    for (int i = 1; i <= 50; i++) {
        value = value + step;
    }
    printf("value = %d\n", value);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int s1 = 1, s2 = 2;
    pthread_create(&t1, NULL, worker, &s1);
    pthread_create(&t2, NULL, worker, &s2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

آیا ممکن است یکی از مقادیر چاپ‌شده در خروجی کمتر از ۶۰ باشد؟ دلیل خود را با تحلیل دستورات در سطح ثبات‌ها[1] و تغییر زمینه[2] به طور کامل توضیح دهید.

ب– آیا ممکن است پس از اتمام هر دو ریسه[3]، مقدار نهایی متغیر `value` برابر با ۱۱ باشد؟ در صورتی که پاسخ مثبت است، نحوه‌ی تولید آن (سناریوی تغییر زمینه بین دو ریسه) را توضیح دهید و در غیر این صورت، کمترین مقدار ممکن برای `value` در پایان برنامه را به دست آورید.

ج– حال فرض کنید بخش ایجاد ریسه‌ها در تابع `main` قطعه کد بالا را به صورت زیر تغییر داده‌ایم:

```c
pthread_t t[2];
int s;
for (s = 1; s <= 2; s++) {
    pthread_create(&t[s - 1], NULL, worker, &s);
}
pthread_join(t[0], NULL);
pthread_join(t[1], NULL);
```

حتی اگر فرض کنیم دستور `value = value + step` هیچ شرایط رقابتی[4] نداشته باشد و به صورت کاملاً اتمیک[5] اجرا شود، با اجرای این کد چه مقادیری ممکن است در نهایت در متغیر `value` ذخیره شود؟ علت جواب خود را با توجه

[1] Registers
[2] Context Switch
[3] Thread
[4] Race Condition
[5] Atomic

به نحوه‌ی اشتراک‌گذاری حافظه میان ریسه‌ها و تفکیک بخش‌های داده[6] و پشته[7] به طور کامل شرح دهید.

دـ اگر در قطعه کد قسمت (ج)، دستور `pthread_join(t[s - 1], NULL)` را به درون حلقه‌ی `for` (بلافاصله پس از `pthread_create`) منتقل کنیم، مقدار نهایی `value` چه خواهد بود؟ دو قطعه کد را با هم مقایسه کنید و علت تفاوت را شرح دهید.

هـ برای اینکه در قطعه کد قسمت (ج) بدون انتقا `pthread_join` به درون حلقه (یعنی با حفظ اجرای موازی دو ریسه)، تضمین شود که هر ریسه مقدار `step` صحیح خود را دریافت می‌کند، چه راهکاری پیشنهاد می‌کنید تا در کد اصلاح شود؟

۲. آـ یک سرور[8] درخواست‌هایش [9] را با یک استخر ریسه[10] اجرا می‌کند:

- استخر **۴ ریسه**[11] و یک صف[12] کار دارد.
- `pool_submit(f, x)` کار `f(x)` را ته صف می‌گذارد.
- `pool_wait(t)` صبر می‌کند تا کار `t` تمام شود.
- هر ریسه‌ای که بیکار باشد، کار اول صف را برمی‌دارد و تا آخر اجرا می‌کند.
- همه‌ی درخواست‌ها از اول در صف هستند.

```
void *part(void *arg) {
    compute(arg);
    return NULL;
}

void *request(void *arg) {
    task_t a = pool_submit(part, arg);
    task_t b = pool_submit(part, arg);
    pool_wait(a);
    pool_wait(b);
    return NULL;
}
```

اگر ۳ درخواست در صف باشد، همه‌ی درخواست‌ها تمام می‌شوند؟ با ۸ درخواست چطور؟

بـ حالا فرض کنید `part` هم مثل `request` رفتار کند؛ یعنی کارش را به دو تکه‌ی کوچک‌تر بشکند، هر دو را به استخر بدهد و منتظر بماند تا تمام شوند. این تکه‌ها دیگر شکسته نمی‌شوند و فقط محاسبه می‌کنند. اگر فقط **یک** درخواست در صف باشد، استخر دست‌کم چند ریسه لازم دارد تا این درخواست همیشه تمام شود، به هر ترتیبی که ریسه‌ها کارها را از صف بردارند؟

جـ در کد زیر، متغیر `a` با مشخصه‌ی `__thread` تعریف شده است که یعنی هر ریسه نسخه‌ی جداگانه‌ای از آن دارد که در آغاز مقدارش صفر است. به این ویژگی، حافظه‌ی محلی ریسه[13] می‌گویند. خروجی برنامه را بنویسید و توضیح دهید مقدار هر متغیر در چهار فراخوانی تابع چگونه تغییر می‌کند. فرض کنید همه‌ی فراخوانی‌های کتابخانه موفق‌اند.

```
__thread int a = 0;
int b = 0;

```

[6] Data Segment
[7] Stack
[8] Server
[9] request
[10] Thread Pool
[11] Thread
[12] Queue
[13] Thread-Local Storage

```
void *f(void *arg) {
    a++;
    b++;
    printf("a=%d b=%d\n", a, b);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, f, NULL);
    pthread_join(t1, NULL);
    pthread_create(&t2, NULL, f, NULL);
    pthread_join(t2, NULL);
    f(NULL);
    f(NULL);
}
```

د- وقتی تابعی مانند open با خطا روبه‌رو می‌شود، کد خطا را در errno قرار می‌دهد. برنامه می‌تواند پس از تشخیص شکست تابع، این مقدار را بخواند تا علت خطا را بفهمد. اگر errno یک متغیر سراسری معمولی بود، در برنامه‌ای با دو ریسه چه مشکلی پیش می‌آمد؟ با یک ترتیب اجرای مشخص، مشکل را نشان دهید و با استفاده از ایده‌ی بند ج راه‌حل بدهید.

۳. پیاده‌سازی و ساختار درونی ریسه‌ها در هسته‌ی سیستم‌های عامل را با تمرکز بر دو رویکرد متفاوت در ویندوز و لینوکس در نظر بگیرید:

- در سیستم‌عامل ویندوز، مدیریت ریسه‌ها بر عهده‌ی سه ساختار سلسله‌مراتبی ETHREAD[۱۴]، KTHREAD[۱۵] و TEB[۱۶] است.
- در لینوکس، تفکیک ساختاری بنیادینی میان پردازه و ریسه در هسته وجود ندارد و هر دو به عنوان یک وظیفه[۱۷] در ساختار داده‌ی task_struct بازنمایی شده و از طریق فراخوانی سیستمی ()clone ایجاد می‌شوند.

با توجه به ساختار این دو سیستم‌عامل به پرسش‌های زیر پاسخ دهید:

آ- سه ساختار داده‌ی ETHREAD، KTHREAD و TEB در ویندوز هرکدام در کدام بخش از حافظه (فضای کاربر یا فضای هسته) مستقر هستند؟ چرا طراحان ویندوز ساختار TEB را بر خلاف دو ساختار دیگر در فضای کاربر قرار داده‌اند؟ اگر داده‌های موجود در این ساختار درون فضای هسته نگهداری می‌شد، چه اثری بر کارایی برنامه‌های چندریسمانی می‌گذاشت؟

ب- در معماری ویندوز، هر ریسه دارای دو پشته‌ی کاملاً مجزا است: «پشته‌ی سطح کاربر» و «پشته‌ی سطح هسته».

۱. لزوم وجود دو پشته‌ی جداگانه را از دیدگاه امنیت و تفکیک سطوح دسترسی توضیح دهید. چرا هسته هنگام اجرای یک فراخوانی سیستمی نمی‌تواند به جای پشته‌ی هسته، فریم‌های خود را روی همان پشته‌ی کاربر ریسه قرار دهد؟

۲. اشاره‌گر به هر یک از این دو پشته در کدام ساختار از میان ساختارهای سه‌گانه‌ی ویندوز نگهداری می‌شود؟

ج- زمان‌بندی و تغییر زمینه‌ی ریسه‌ها در ویندوز بر اساس تفکیک لایه‌ای سیستم‌عامل انجام می‌گیرد:

۱. هنگام تعویض پردازنده میان دو ریسه از یک پردازه‌ی مشترک، زمان‌بند هسته مستقیماً با کدام ساختار داده (ETHREAD یا KTHREAD) تعامل دارد؟

[14] Executive Thread Block
[15] Kernel Thread Block
[16] Thread Environment Block
[17] Task

۲. هدف معماری ویندوز از تفکیک اطلاعات ریسه به دو بخش مجزای لایه‌ی اجرایی[۱۸] و لایه‌ی هسته[۱۹] چیست؟

د- لینوکس بر خلاف ویندوز از یک مدل یکپارچه استفاده می‌کند:

۱. زمان‌بند هسته‌ی لینوکس چگونه با موجودیت‌های اجرایی برخورد می‌کند و مفهوم «ریسه» چگونه در این مدل تعریف می‌شود؟
۲. اگر برنامه‌ای شامل یک پردازه با ۴ ریسه را در نظر بگیریم، هسته‌ی ویندوز و هسته‌ی لینوکس به ترتیب چه ساختارهای داده‌ای را در فضای هسته برای این برنامه ایجاد و نگهداری می‌کنند؟

ه- فرض کنید در لینوکس، یک وظیفه‌ی جدید با فراخوانی `clone(child_func, stack, CLONE_VM | CLONE_FILES, NULL)` ایجاد شده باشد. وضعیت هر یک از موارد زیر را پس از اجرا در وظیفه‌ی فرزند بررسی کنید:

۱. اگر فرزند فایلی را با `open()` باز کند یا توصیف‌گر شماره ۳ را با `close(3)` ببندد، آیا تغییرات در والد منعکس می‌شود؟ چرا؟
۲. اگر فرزند پوشه‌ی کاری را با `chdir("/tmp")` تغییر دهد، آیا پوشه‌ی کاری والد تغییر می‌کند؟ برای اشتراک این مورد کدام پرچم لازم است؟
۳. اگر فرزند تابع جدیدی برای رسیدگی به `SIGINT` ثبت کند، آیا واکنش والد تغییر می‌کند؟ کدام پرچم مسئول این اشتراک است؟

و- رویکرد انعطاف‌پذیر لینوکس بر پایه‌ی پرچم‌های `clone()` را با رویکرد صلب و ساختارهای سلسله‌مراتبی ویندوز مقایسه کنید. هر رویکرد چه مزیتی در طراحی سیستم‌عامل ارائه می‌دهد؟ (به کاربردهایی نظیر ساخت کانتینر در لینوکس و تفکیک لایه‌ای در ویندوز اشاره کنید).

۴. به سوالات زیر پاسخ دهید:

آ- کد زیر را در نظر بگیرید.

```
int global;
void* foo(void* arg) {
    printf("%p\n", &foo);
    printf("%p\n", &global);
    printf("%p\n", &arg);
    printf("%p\n", arg);
    return NULL;
}
int main() {
    void* hmem = malloc(1);
    for (int i = 0; i < 3; i++) {
        pthread_t pid;
        pthread_create(&pid, NULL, foo, hmem);
    }
}
```

از بین خطوط `printf` داده‌شده، کدام موارد خروجی یکسانی در همه ریسه‌ها `Thread` می‌دهند؟ توضیح دهید.

ب- کد زیر را در نظر بگیرید که می‌تواند توسط یک یا چند ریسه اجرا شود:

```
x = x + 1
y = x + y
```

فرض کنید:

- قبل از شروع هر ریسه‌ای، متغیرهای y و x مقدار اولیه‌ی ۱ را دارند. پس از آن فقط ریسه‌های در حال اجرای کد بالا می‌توانند مقادیر x و y را تغییر دهند.

[18] Executive Layer
[19] Kernel Layer

- هر دو دستور atomic هستند.
- هر یک از ریسه‌ها در هر لحظه از اجرا ممکن است متوقف شود و اجرا را واگذار کند.

۱. فرض کنید ۲ کپی از کد بالا به طور هم‌روند با ۲ ریسه اجرا می شود. تمام مقادیر ممکن برای x و y را پس از اتمام کار هر ۲ ریسه بنویسید.
۲. سوال قبل را به ازای ۳ ریسه پاسخ دهید.
۳. فرض کنید n کپی از کد بالا به طور هم‌روند با n ریسه اجرا می شود. حداکثر مقداری که متغیر y پس از اتمام اجرای تمامی ریسه‌ها می‌تواند داشته باشد را محاسبه نمایید.
۴. مسئله قبل را به ازای حداقل مقدار y پاسخ دهید.

تمارین عملی

۱. این تمرین دربارهٔ **نخ‌ها**[20] است و در ادامهٔ تمرین‌های قبلی، فقط در **فضای کاربر** و بدون هیچ تغییری در هسته انجام می‌شود. برخلاف فرایندها که هر کدام حافظهٔ جداگانه دارند، نخ‌های یک فرایند کد و دادهٔ سراسری را با هم شریک‌اند و هر نخ فقط پشته و ثبات‌های خودش را دارد. در این تمرین یک کتابخانهٔ نخِ تعاونی[21] برای xv6 می‌نویسید: بدون اینکه هسته چیزی از نخ‌ها بداند، خودِ برنامه با یک تابع جابه‌جایی زمینه[22] چند جریان اجرا را به نوبت اجرا می‌کند؛ همان ایدهٔ swtch در هسته، اما این بار در سطح کاربر. برنامه‌ریزی هسته و هم‌زمانی با قفل و سمافور موضوع درس‌های بعدی است و در این تمرین به آن‌ها نمی‌پردازیم.

پیش‌نیازها همان تمرین قبل است و محیط اجرا نیازی به تنظیم مجدد ندارد. در پوشهٔ starter اسکلت دو فایل user/uthread.c و user/uthread_switch.S با بخش‌های TODO داده شده است؛ آن‌ها را در مخزن xv6 خود کپی کنید و تکمیل کنید. بخش‌های worker0، worker1، worker2 و main را تغییر ندهید.

قواعد این کتابخانه به این صورت است:

- هر نخ یک پشتهٔ 8 کیلوبایتی دارد و در context آن فقط ثبات‌های callee-saved یعنی ra، sp و s0 تا s11 ذخیره می‌شوند. دلیلش این است که yield یک فراخوانی تابع معمولی است و ثبات‌های caller-saved را خودِ فراخواننده حفظ می‌کند.
- برنامه چهار جایگاه نخ دارد: جایگاه 0 خودِ main است و جایگاه‌های 1 تا 3 برای سه نخِ کارگر.
- هر نخ سه بار پیام خود را چاپ می‌کند و بین هر دو بار yield می‌کند؛ در پایان هم پیام پایان خود را چاپ می‌کند.

خروجی برنامه باید دقیقاً به این شکل باشد:

```
uthread: thread 0 tick 0
uthread: thread 1 tick 0
uthread: thread 2 tick 0
uthread: thread 0 tick 1
uthread: thread 1 tick 1
uthread: thread 2 tick 1
uthread: thread 0 tick 2
uthread: thread 1 tick 2
uthread: thread 2 tick 2
uthread: thread 0 finished
uthread: thread 1 finished
uthread: thread 2 finished
uthread: done
```

۱. تابع thread_create(func) را پیاده‌سازی کنید. این تابع اولین جایگاه آزاد (FREE) را پیدا می‌کند، وضعیت آن را RUNNABLE می‌گذارد، ra را برابر func و sp را برابر بالای پشتهٔ همان نخ قرار می‌دهد و در صورت موفقیت 0 برمی‌گرداند. اگر جای خالی نبود، 1- برگردانید.

۲. توابع thread_yield() و thread_schedule() را پیاده‌سازی کنید. thread_yield() نخِ جاری را RUNNABLE علامت می‌دهد و سپس thread_schedule() را فراخوانی می‌کند. thread_schedule() نخِ آمادهٔ بعدی را به صورت round-robin و دقیقاً از نخِ بعدیِ نخِ جاری (با چرخش دورانی) انتخاب می‌کند؛ اگر هیچ نخی آماده نبود، کنترل باید به نخِ اصلی یعنی جایگاه 0 برگردد (اگر خودِ نخِ جاری main باشد، thread_schedule() فقط برمی‌گردد). برای انجام جابه‌جایی، thread_switch را با آدرس context نخِ جاری و نخِ بعدی صدا بزنید.

[20] threads
[21] cooperative
[22] context switch

۳. تابع thread_switch را در فایل user/uthread_switch.S به زبان اسمبلی پیاده‌سازی کنید. این تابع دو آرگومان دارد: a0 آدرس context نخِ جاری (برای ذخیره) و a1 آدرس context نخِ بعدی (برای بارگذاری). ثبات‌های ra، sp و s0 تا s11 را در context نخِ جاری ذخیره کنید، سپس همان‌ها را از context نخِ بعدی بارگذاری کنید و با ret به نخِ بعدی برگردید. چیدمان context همان ترتیب فیلدهای struct thread_context در uthread.c است (ra در آفست 0، sp در 8 و سپس s0 تا s11 هر 8 بایت یکی).

نکتهٔ ساخت: برای اینکه xv6 این برنامه را بسازد، باید user/_uthread را به UPROGS در Makefile اضافه کنید و یک قاعده برای اسمبل کردن فایل S. بنویسید. آزمون خودکار در صورت فراموشی این دو مورد را خودش اضافه می‌کند، اما بهتر است خودتان دلیلش را بفهمید.

سؤال ویدیو: چرا فقط ثبات‌های callee-saved ذخیره می‌شوند و نه همهٔ ثبات‌ها؟ و چرا ترتیب دقیقِ خطوط خروجی حتمی است؟ در ویدیو به این سؤال پاسخ دهید.

آزمون خودکار

همراه این تمرین پوشه hw3-tests داده شده است. آن را در ریشه مخزن xv6 کپی کنید و با دستور زیر خودتان را بسنجید:

```
python3 hw3-tests/grade-hw3.py
```

این اسکریپت در صورت نیاز ورودی uthread_ و قواعد ساخت فایل S. را به Makefile اضافه می‌کند، xv6 را می‌سازد، در QEMU بوت می‌کند و دستور uthread را اجرا می‌کند؛ سپس ترتیب دقیق سیزده خط خروجی را بررسی می‌کند. خروجی نهایی باید score: 1/1 باشد و اسکریپت همان ابزاری است که برای نمره‌دهی استفاده می‌شود.

خروجی‌های قابل تحویل

این تمرین به صورت گروهی (دو نفره) انجام می‌شود و خروجی آن شامل دو بخش است:

۱. یک آرشیو فشرده (zip.) با نام hw3-lastname1-lastname2.zip شامل:
- فایل‌های تکمیل‌شده user/uthread.c و user/uthread_switch.S و Makefile
- اسکرین‌شات خروج کامل آزمون خودکار (score: 1/1)

۲. یک ویدیوی حداکثر 6-5 دقیقه‌ای که در آن موارد زیر را نمایش دهید:
- اجرای uthread و توضیح اینکه چرا نخ‌ها به این ترتیب اجرا می‌شوند
- توضیح اینکه در context چه ثبات‌هایی ذخیره شده‌اند و چرا

در ویدیو مراحل انجام‌شده را به صورت کوتاه توضیح دهید و نیازی به توضیح خط‌به‌خط کدها نیست.

لنگر: جداسازی پردازه و نام میزبان

در این مرحله، برنامهٔ اجراشده در لنگر، شناسهٔ پردازه و نام میزبان مستقلی خواهد داشت. این جداسازی را به کد تمرین دوم خود اضافه می‌کنید؛ اجرای معمولی تمرین قبل باید همچنان کار کند.

محیط و فایل‌های اولیه یک کرنل لینوکس بومی یا ماشین مجازی درس لازم است. ساخت با کاربر عادی انجام می‌شود؛ آزمون جداسازی و mount در محیط آزمایش به دسترسی root نیاز دارد. در macOS و Windows از backend ماشین مجازی Langar Lab استفاده کنید؛ در Linux می‌توانید native را انتخاب کنید. فایل‌ها را از نسخهٔ مربوط به همین تمرین بگیرید:

```
https://github.com/MohammadAminKoohi/OS-Practical
Release tag: hw03-v1.0.0
```

تغییرهای تجزیه‌گر و helperها را با کد خود ادغام کنید. run.c اولیه عمداً حل نشده است؛ آن را جای پاسخ کامل هفتهٔ قبل ننشانید. پیاده‌سازی اصلی همچنان با C۱۱ و گزینه‌های Wextra Wall، و Werror ساخته شود؛ استفاده از popen system، و جایگزین کردن runtime با Docker مجاز نیست. اجرای خودکار sh -c به جای argv اصلی مجاز نیست؛ shell در مثال‌ها فقط وقتی اجرا می‌شود که کاربر صریحاً آن را به‌عنوان COMMAND داده باشد.

```
./langar run -- /bin/echo hello
sudo ./langar run --isolate --hostname team1 -- /bin/hostname
sudo ./langar run --isolate -- /bin/sh -c 'echo $$'
```

جداکنندهٔ — الزامی است. تجزیه‌گر برای گزینهٔ ناشناخته، گزینهٔ تکراری، فرمان ناقص یا ترکیب نامعتبر، پیام stderr و کد ۲ می‌دهد. خطای setup کد ۱ دارد. خطاهای exec و وضعیت دستور، همان قرارداد تمرین دوم هستند: دستور ناموجود ۱۲۷، سایر خطاهای exec برابر ۱۲۶، خروج عادی همان مقدار و مرگ با سیگنال برابر signal+۱۲۸. خطای wait یا fork نیز ۱ است. پیام اضافه در stdout چاپ نکنید.

کار خواسته‌شده

۱. رابط تابع اجرای خود را مطابق سرآیند جدید به `run_command(argv, options)` تغییر دهید. منطق fork/exec/wait تمرین قبل را نگه دارید؛ فایل اولیهٔ جدید پاسخ تمرین قبل را پیاده‌سازی نکرده است.

۲. با فعال بودن `--isolate`، یک فضای نام PID بسازید. توجه کنید `unshare(CLONE_NEWPID)` فضای نام فرزندهای بعدی را عوض می‌کند؛ خود فراخواننده فوراً پردازهٔ شمارهٔ یک نمی‌شود. دستور باید در فضای نام جدید، PID=1 و PPID=0 مشاهده کند و لنگر در بیرون منتظر آن بماند.

۳. برای همان اجرای ایزوله، فضای نام UTS ایجاد و با `sethostname()` نام میزبان را تنظیم کنید. گزینهٔ `--hostname` NAME فقط همراه جداسازی مجاز است. در نبود آن نام پیش‌فرض `langar` باشد. نام میزبان سیستم میزبان نباید تغییر کند.

۴. رفتار ورودی، خروجی، آرگومان‌ها، محیط، پوشهٔ کاری و کد خروج تمرین قبل را نگه دارید. اجرای بدون گزینهٔ جداسازی، همان قرارداد قبلی و یک فرزند مستقیم را دارد.

مرز این مرحله در این مرحله فضای نام mount و اتصال تازهٔ procfs لازم نیست. بنابراین از خروجی ps برای اثبات جداسازی PID استفاده نکنید: proc/ موروثی هنوز نمای میزبان را دارد. داور با برنامهٔ C و بررسی هویت namespace آزمایش می‌کند. پردازهٔ PID 1 قواعد ویژهٔ سیگنال دارد؛ ارسال SIGTERM به اجرای ایزوله در این هفته خواسته نشده است. آزمون سیگنال عادی تمرین دوم فقط روی اجرای بدون جداسازی انجام می‌شود.

آزمون و ارزیابی مبنای نمرهٔ عملکرد کد، آزمون‌های مستقل داور با مجموع ۱۰۰ امتیاز است. وزن هر قابلیت در JUDGE.md همین نسخه آمده است. بخشی از امتیاز به سالم ماندن قابلیت‌های قبلی اختصاص دارد. داور تنها رفتارهای تصریح‌شده را می‌سنجد؛ ورودی‌های نهایی می‌توانند با نمونه‌ها فرق کنند. نبود قابلیت هسته یا مجوز namespace خطای محیط است و نمرهٔ عملکرد محسوب نمی‌شود. گزارش و ویدیو نیز برای بررسی توضیح و الزامات کد لازم‌اند و امتیاز آزمون جای تحویل آن‌ها را نمی‌گیرد.

```
langar-lab test hw3 --source ./q2 --json ./result.json
# Directly inside Linux:
make -C q2 clean
make -C q2 CC=gcc
sudo python3 judge/test_semester.py --assignment 3 --binary ./q2/langar --json result.json
```

پرسش‌های گزارش پاسخ‌ها کوتاه و با ارجاع به کد و خروجی واقعی خودتان باشند.

۱. چرا پس از unshare مربوط به PID هنوز به fork نیاز دارید؟ ترتیب واقعی این دو فراخوان را در کدتان نشان دهید.
۲. کدام اطلاعات این هفته جدا شده‌اند و کدام منابع، مانند فایل‌ها و شبکه، هنوز مشترک هستند؟

ویدیوی پاسخ و توضیح کد در ویدیویی حدود پنج تا هشت دقیقه، محل تغییرهای مهم نسبت به هفتهٔ قبل، یک اجرای موفق، یک خطای مربوط به قابلیت جدید و نتیجهٔ داور را نشان دهید. خواندن خط‌به‌خط کل برنامه لازم نیست. علاوه بر توضیح کد، به هر دو پرسش زیر پاسخ دهید:

۱. چگونه ممکن است getpid عدد ۱ نشان دهد ولی ابزار ps با proc/ موروثی نمای دیگری نشان دهد؟
۲. اگر sethostname پیش از ایجاد UTS namespace اجرا شود چه منبعی ممکن است تغییر کند؟ کد شما چگونه از آن جلوگیری می‌کند؟

ساختار تحویل پوشه q2 را مستقیم در ریشه فایل فشرده‌شده خود قرار دهید. همچنان توجه کنید که کد کامل این نسخه را تحویل دهید نه فقط diff یا فایل تغییر کرده. ساختار الزامی:

```
q2/
  Makefile
  include/runner.h
  src/main.c
  src/run.c
  answers.md
  test-results.txt
  video.mp4
```

فایل answers.md شامل نسخهٔ کرنل و معماری، پاسخ‌های شماره‌گذاری‌شده، محدودیت‌های شناخته‌شده و زمان شروع پاسخ دو پرسش ویدیو باشد. test-results.txt خروجی واقعی آخرین داور روی همین نسخه است. ویدیو فایل MP4 خوانا باشد؛ برای کاهش حجم می‌توانید ffmpeg به کار ببرید. فایل کمکی C و سرآیند اضافی در src و include مجاز است. فایل اجرایی و object، git. آزمون‌های درس و rootfs یا ایمیج دانلودشده را داخل آرشیو نگذارید. تمام متن‌ها UTF-۸ باشند. ساخت باید بدون شبکه، نصب بسته، ورودی تعاملی یا root انجام شود و هدف پیش‌فرض Makefile فایل q2/langar را بسازد. متغیر CC و هدف clean حفظ شوند. برنامه را از هر پوشهٔ فراخوان قابل اجرا نگه دارید؛ مسیر مطلق رایانهٔ خود را در آن ننویسید. داور از آزمون‌های مورد اعتماد درس استفاده می‌کند و فایل نمرهٔ ارسالی شما را مرجع تصحیح قرار نمی‌دهد.

منابع اصلی برای ترتیب و رفتار دقیق فراخوان‌ها، مستندات زیر را بخوانید.

- unshare(2)
- PID namespaces
- UTS namespaces
- sethostname(2)